Terminal × ascii_art.py

```
(base) codio@banditruby-questbeyond:~/workspace$ python ascii-flask/ascii_art.py
Please enter a valid URL for an image
Please enter a valid URL for an image
```

Guide ×

Collapse

Creating ASCII Art

Setting Up Our Virtual Environment

ASCII art is a representation of words or images using only ASCII characters. In this project, we will take the URL of an image and return an ASCII art rendition of the image. Before we can do this, we need to setup our work space. Create the `ascii` virtual environment with Conda.

```
conda create --name ascii -y
```

Then activate the environment.

```
conda activate ascii
```

Next, install `pillow` an image manipulation library for Python. This needs to be installed from the Conda Forge channel.

```
conda install pillow -c conda-forge -y
```

We also need to install the `requests` package.

```
conda install requests -y
```

Finally, open the `ascii_art.py` file. This is where we will create an algorithm to convert an image from the internet into a list of strings. Each character in the string represents a pixel from the original image.

Open ascii_art.py

Transforming an Image into ASCII Art

Start by importing the necessary packages. `PIL` is the Pillow package. The `requests` package is used to fetch data from a website. `BytesIO` is used to store the contents of the image from the internet.

```
from PIL import Image
import requests
from io import BytesIO
```

Define the `create_ascii` function which takes the URL of an image. We need to be able to handle bad input from the user. If not, the Flask app will crash when it cannot open an image URL. There are two main ways in which the app would crash. One, the URL is not given in its full form - `https://...`. Two, the URL provided is not the URL for an image. This project assumes users will enter a URL for an image; i.e., if you enter that URL in a browser an image (and only an image) would load. If we cannot open the webpage or convert it to an image, return a string telling the user to enter a valid URL.

```
def create_ascii(img_url):
  try:
    response = requests.get(img_url)
    pic = Image.open(BytesIO(response.content))
  except:
    return 'Please enter a valid URL for an image'
```

We need to resize the image. Unpack the `width` and `height` from the `pic.size` tuple. Figure out the aspect ratio for the image. Create a new width of `110` and using the aspect ratio, give the image a new height. Resize the image to the new width and height.

```
  width, height = pic.size
  aspect_ratio = height / width
  new_width = 110
  new_height = int(aspect_ratio * new_width)
  img = pic.resize((new_width, new_height))
```

Convert the image to grayscale. Then use the `getdata()` method to get the data for each pixel in the image. This returns a list of numbers each with a value from 0 to 255. 0 represents the color black and 255 represents the color white. Numbers in between are various shades of gray. Create the list `chars`. The first character should have the "most writing" in it. As you move through the list, the characters have less and less "writing". At the very end, you have a blank space. The progression of characters matches the progression from 0 (black) to 255 (white). Then count the number of characters in the list `chars`.

```
  img = img.convert('L')
  pixels = img.getdata()
  chars = ['@', '#', 'S', '%', '?', '*', '+', ';', ':', ',', '.', ' ']
  count = len(chars)
```

This is the most important part of the ASCII art algorithm. For each pixel in the image, find the corresponding character in the `chars` list. Then, use the `join` method to take the list of strings and turn it into one giant string. Calculate the length of this giant string. We need to insert line breaks every 110 characters (the new width of our ASCII art). Create the list `ascii_image`. Using a comprehension, loop over the range from 0 to the length of the `new_pixels` string. Use the interval of `new_width` which is 110. This means the index of the loop is 0, 110, 220, etc. until the end of the string. Use the slice operator to take each chunk of 110 characters and put them in `ascii_image`. Lastly, return the `ascii_image` list.

```
  new_pixels = [chars[int(((count-1)*pixel)/256)] for pixel in pixels]
  new_pixels = ''.join(new_pixels)
  new_pixels_count = len(new_pixels)
  ascii_image = [new_pixels[index:index + new_width] for index in range(0, new_pixe
  return ascii_image
```

Finally, we need to test our code. Add a conditional that checks if we are running `ascii_art.py` directly. Start our testing with the two examples of a bad URL. The first URL (`google.com`) is bad because it is missing `https://...`. The second example is bad because the URL is for a valid webpage, but it is not an image URL. Finally, test the `create_ascii` function with a working URL. Iterate over the list, printing each string in the list.

```
if __name__ == '__main__':
  print(create_ascii('google.com'))
  print(create_ascii('https://www.google.com/'))
  art = create_ascii('https://images.pexels.com/photos/5302784/pexels-photo-5302784
  for line in art:
    print(line)
```

Swtich back to the tab with the terminal and enter the following command. You should see two messages about entering a valid image URL as well as the silhouette of an individual printed in the terminal.

Open terminal

```
python ascii-flask/ascii_art.py
```

Deactivate the virtual environment.

```
conda deactivate
```

▼ **Code**

Your code should look like this:

```
from PIL import Image
import requests
from io import BytesIO

def create_ascii(img_url):
  try:
    response = requests.get(img_url)
    pic = Image.open(BytesIO(response.content))
  except:
    return 'Please enter a valid URL for an image'

  width, height = pic.size
  aspect_ratio = height / width
  new_width = 110
  new_height = int(aspect_ratio * new_width)
  img = pic.resize((new_width, new_height))
  img = img.convert('L')
  pixels = img.getdata()
  chars = ['@', '#', 'S', '%', '?', '*', '+', ';', ':', ',', '.', ' ']
  count = len(chars)
  new_pixels = [chars[int(((count-1)*pixel)/256)] for pixel in pixels]
  new_pixels = ''.join(new_pixels)
  new_pixels_count = len(new_pixels)
  ascii_image = [new_pixels[index:index + new_width] for index in range(0, new_pixe
  return ascii_image

if __name__ == '__main__':
  print(create_ascii('google.com'))
  print(create_ascii('https://www.google.com/'))
  art = create_ascii('https://images.pexels.com/photos/5302784/pexels-photo-5302784
  for line in art:
    print(line)
```

Lab Question

True of False: Pyinstaller is part of the standard library in Python.

- (•) False
- ( ) True

False. Pyinstaller is a third-party package you have to install from a repository. Therefore it is not a part of the standard library.

Check It!

Next